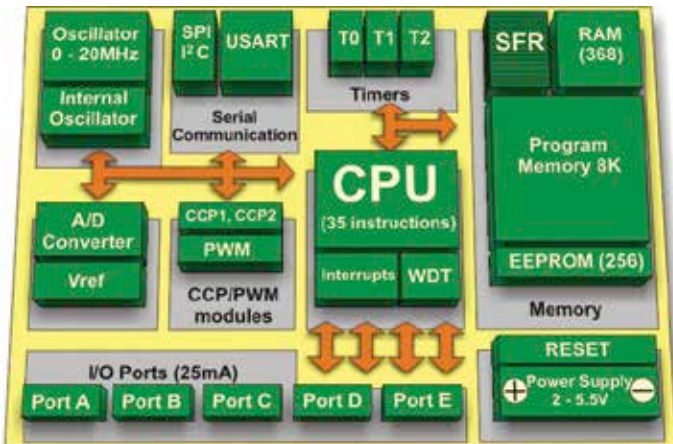


code on your board, now how you do this will again change from platform to platform but for development purposes most boards provide a 'host-target' interface which allows the user to use a normal \*nix/Windows machine and the development version of the board is attached to the host via USB, ethernet, parallel port or RS-232 and the developer can directly download the code into the board and run it there. This changes for production systems where the code is usually burnt into a programmable ROM, which can be easily mass produced.

Another important aspect is debugging, this is where in case of errors in your program check the status of the system step-by-step and zero in on the problem area. The debugging methods also vary from platform to platform, we'll discuss them in detail later. The programmer then fixes the bug builds the program and deploys it again checking for errors and thus the cycle continues, until the code is ready for production.

## Hardware

The first thing that you as a software engineer on an embedded project should do is to get yourself familiarised with the hardware on which the software will run. We can't stress this fact enough, each board has it's own idiosyncrasies, hardware components and some feature that are entirely unique to that board like the timestamp counter and the MMX registers in the Pentium processors which are not available on other Intel boards, let



PIC16F887 Microcontroller block diagram